Chess Server Design

WebTransport over QUIC · one "interface" thread · 10 worker tasks · 100 connections · waiting line

This is a reference for the game server in `Server/`, written to match the code as it is now. It covers four things: how the server is structured, how a client gets in, how a request is handled, and how the server is tested and measured. Every component shown exists in the code; nothing is planned or assumed.

1. Threads and the interface

The `main` thread starts one thread, named `interface`, and waits for it to finish. The `interface` thread runs a tokio **current-thread** runtime inside a `LocalSet`. On it run exactly **11 tasks**: the interface, which accepts and admits clients, and **10 worker tasks**, which handle all 100 connections between them.

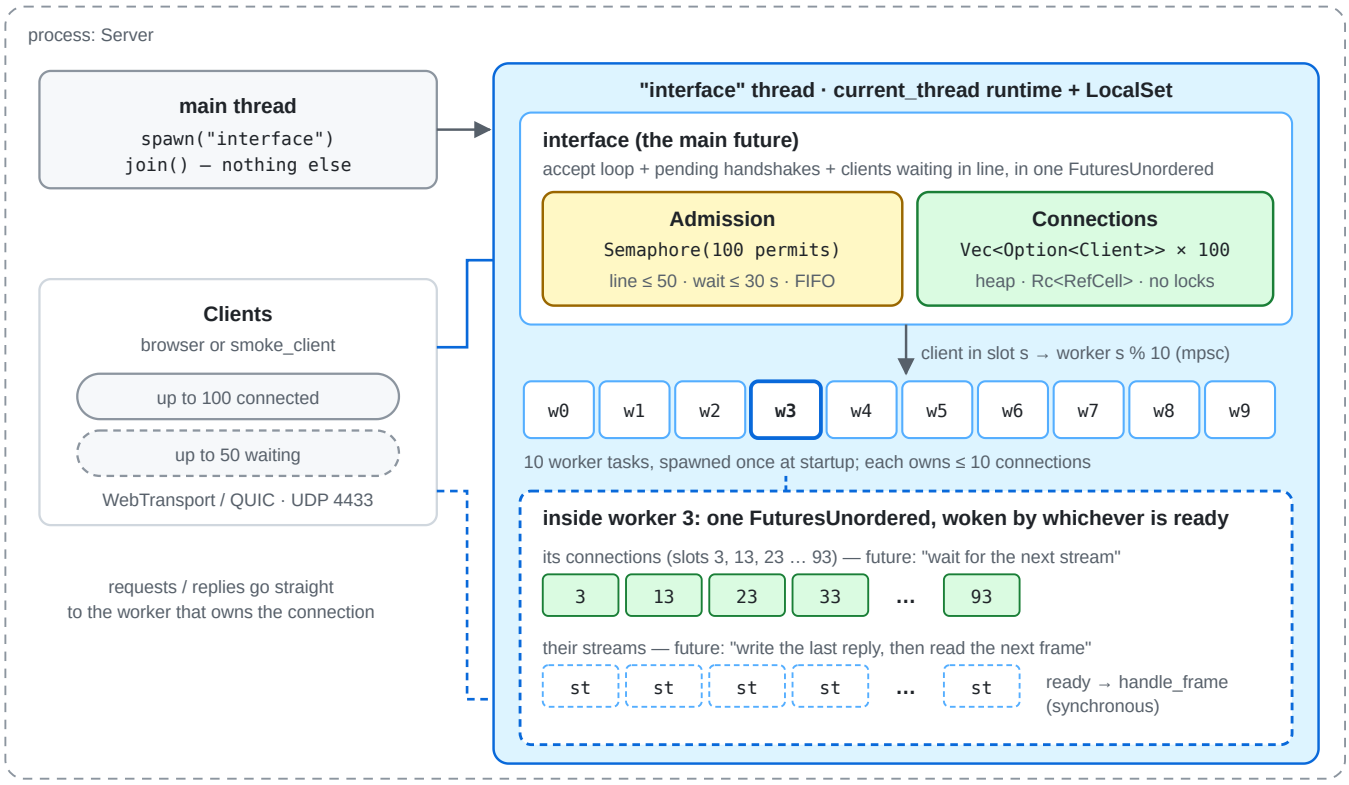


Figure 1. One working OS thread (`interface`) runs the interface future and 10 workers. A client admitted into slot *s* is owned by worker *s* % 10 for its whole life, so each worker owns at most 10 connections. The task count stays at 11 whether 1 or 100 clients are connected. Admission and Connections are shared without locks.

1b. Components and the concurrency model

Part	File	Job
`main`	`src/main.rs`	Starts the `interface` thread and waits for it
`run` (interface)	`src/main.rs`	Binds the endpoint, starts the 10 workers, accepts clients, sends each admitted client to its worker, shuts down on Ctrl-C
`admit`	`src/main.rs`	Handshake → permit (now, or after waiting in line) → slot
`worker::run`	`src/worker.rs`	One of the 10 workers: waits on all its connections and streams, handles whichever is ready
`handle_frame`, `handle_request`	`src/worker.rs`	Plain synchronous code: decode → read or update the client's slot → encode the reply
`Admission`	`src/admission.rs`	100 permits plus a bounded FIFO waiting line
`Connections`	`src/connections.rs`	The 100-slot heap `Vec` of clients
`Payload`	`src/Payload/`	Request/Response encode and decode, shared with the test client

Single thread, but asynchronous

Question	Answer
How many threads do the work?	**One**: `interface`. `main` only waits.
How many tasks?	**11**: the interface plus 10 workers, fixed at startup. (wtransport's QUIC library also runs its own internal tasks on this thread.)
How does one worker serve 10 connections?	It keeps one pending future per connection ("next stream") and one per stream ("write the last reply, then read the next frame") in a `FuturesUnordered`, and handles whichever becomes ready.
Are clients served in parallel?	**No.** Only one piece of code runs at any instant.
Are clients served concurrently?	**Yes.** All 100 connections are in progress at once; the 10 workers and the interface take turns on the thread.
When does a task give up the thread?	When none of its futures is ready. Because writes happen inside each stream's own future, a slow client only holds up its own stream, not the rest of its worker.
Why no locks?	`handle_request` has no `.await`, so one request finishes before the next one starts, and only this thread touches the store.
What would freeze everyone?	Slow synchronous work inside `handle_request`, such as a deep engine search. That kind of work belongs on `spawn_blocking`.

> Every slot stores a unique client `id`, and lookups check the slot index and the id together. A stream left over from a client that has gone can therefore never change the next client that takes the same slot.

2. How a client gets in

Admission is a `tokio::sync::Semaphore` with one permit per slot. Its wait list is the buffer that holds clients until a slot is free. A session holds its permit until the client leaves, so holding a permit guarantees a free slot.

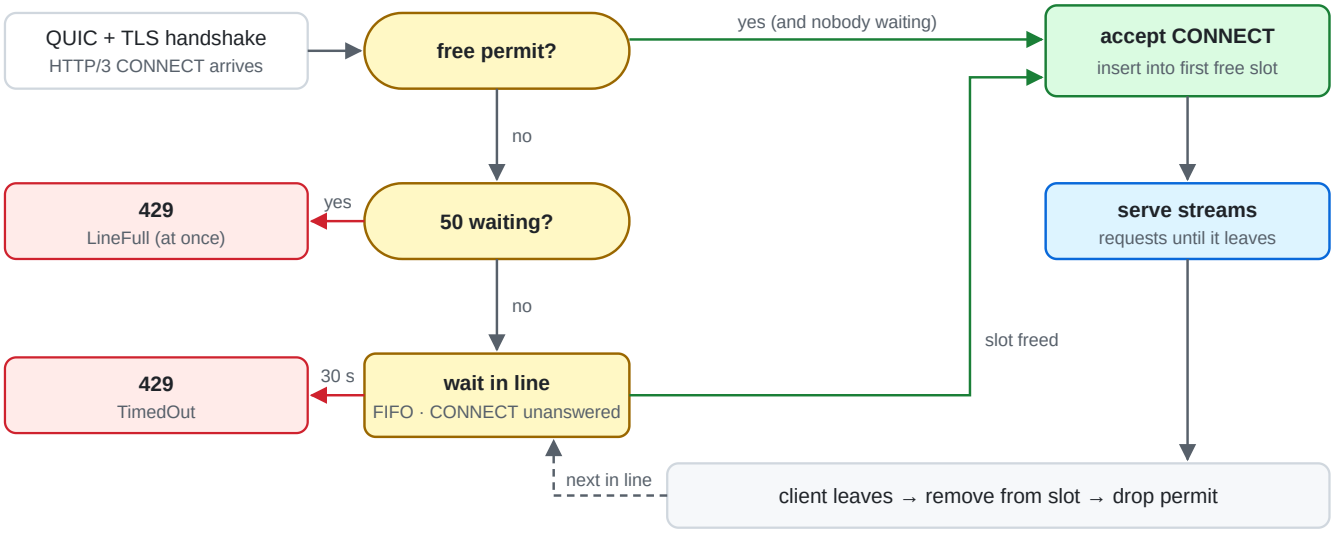


Figure 2. Admission decisions. A waiting client has finished its QUIC handshake but has no answer to its `CONNECT` yet, so a browser shows it as "connecting". The server's 3 s keep-alive keeps its connection open while it waits.

Situation	What happens
A slot is free and nobody is waiting	The client enters immediately
All 100 slots taken, fewer than 50 waiting	The client waits in line
A connected client leaves	Its permit goes straight to the client that has waited longest
50 already waiting	Refused at once with 429 (`LineFull`)
Waited 30 s with no free slot	Refused with 429 (`TimedOut`)
Ctrl-C	Every waiting client is refused (`Closed`), and every connection is closed

The limits are constants: `MAX_CONNECTIONS = 100` in `src/connections.rs` , and `MAX_WAITING = 50` and `MAX_WAIT = 30 s` in `src/admission.rs` .

3. How a request is handled

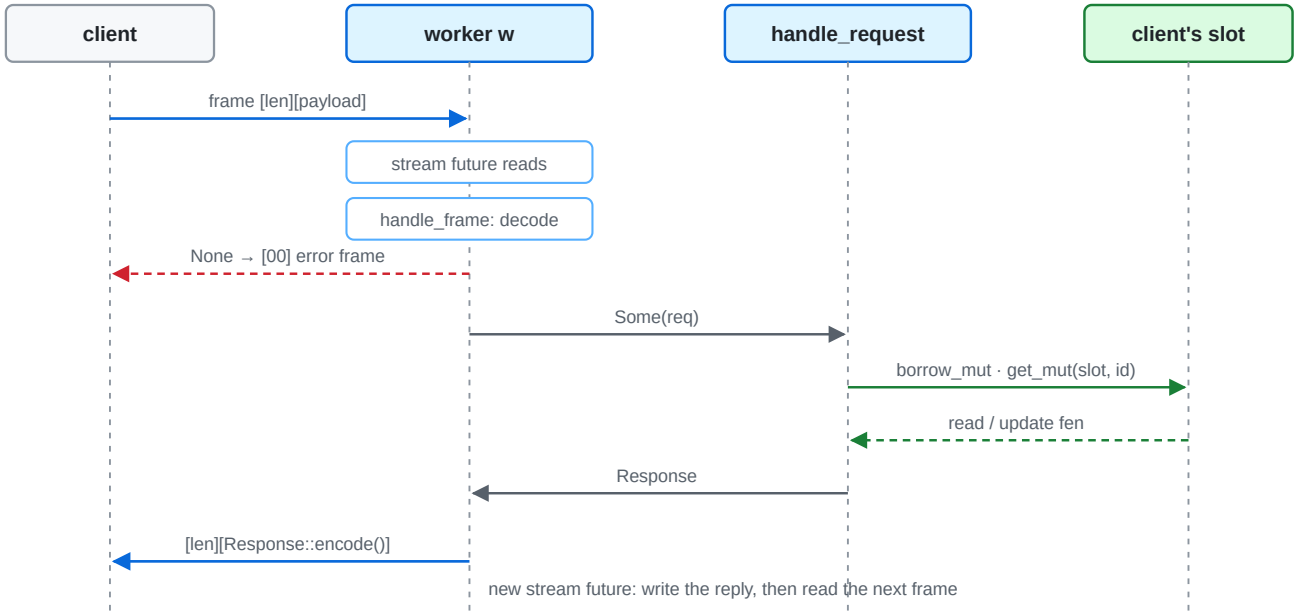


Figure 3. One request on one stream, inside the worker that owns the connection. The stream's future reads the frame and wakes the worker. `handle_frame` runs synchronously, with no `.await`, and puts the reply in the stream's buffer. The next stream future then writes the reply and reads the next frame.

Request	Payload length	Server reply
`GetFen`	1	`Fen` of this client's board (32 bytes)
`SetFen`	32	Stores the board in the slot, replies `Fen` of it
`Select(u64)`	8	`Select` with the same square (an echo until the engine is connected)
`Move`	17	`Move` unchanged (an echo until the engine is connected)
any other length	—	Zero-length error frame; the stream stays open

Wire frame

```
[ len: u8 ][ payload: len bytes ]      same in both directions

len  1 → GetFen         (request)
len  8 → Select(u64)    (request and response), big-endian
len 17 → Move           from u64 + to u64 + role u8, big-endian
len 32 → SetFen / Fen   64 squares × 4 bits (piece << 1 | colour)
len  0 → error frame    (response only)
```

4. How performance is tested

`examples/smoke_client.rs` is a real WebTransport client. It includes the server's own `src/Payload` code through `#[path]`. Its request dispatcher therefore builds requests with `Request::encode` and reads replies with `Response::decode`, the exact inverse of what the server does.

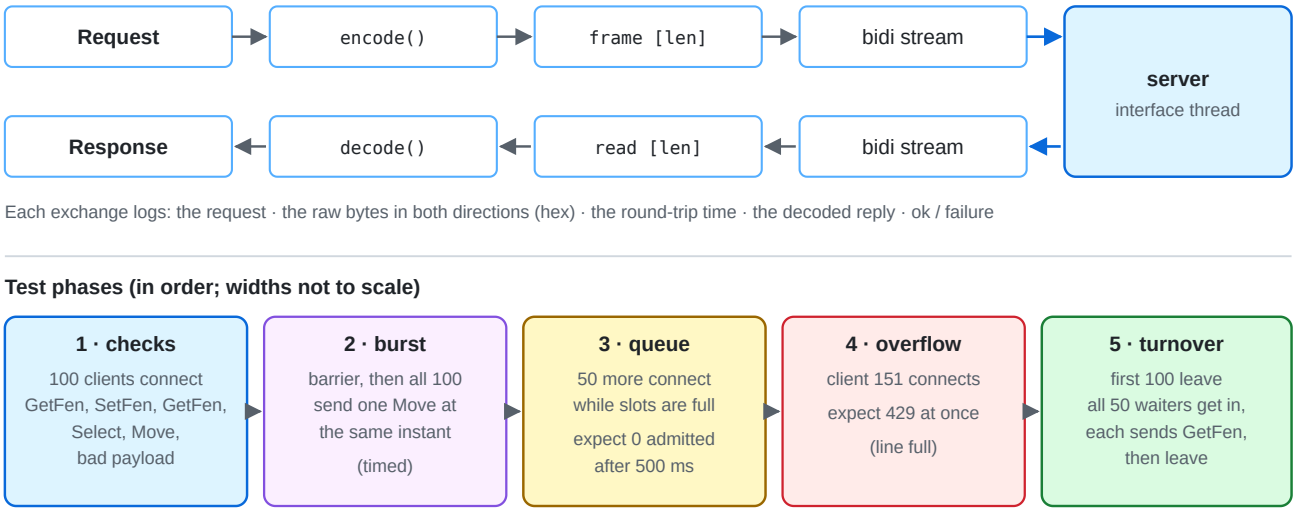


Figure 4. Top: the request dispatcher in the test client. Bottom: the five test phases. Any wrong reply, or any admission behaving differently from expected, stops the test with an error and a non-zero exit code.

Results (localhost, release build, 3 runs, all passed)

Measurement	Result (range over 3 runs)
Burst: 100 Moves at the same instant, all replies	**3.4 – 5.8 ms** (round trip p50 1.8 – 3.5 ms)
Waiters admitted while all slots are full	0 of 50
Client 151 refused	2.4 – 6.2 ms, with 429
After 100 slots freed: 50 waiters served	12 – 22 ms
Requests per worker (one run)	63 – 67 each; all 10 workers busy
Server threads in the log	`interface`: all 1306 lines of client work · `main`: 2 lines (start, finish)

How to run it

```
# terminal 1: the server prints "cert sha-256: …"
cargo run --release
# terminal 2: pass that hash to the test
cargo run --release --example smoke_client -- <cert-sha256>
```

What the test does not cover: a real network (everything runs on one machine, so there is no real latency or packet loss), a real browser client, the 30 s wait timeout, and real chess rules (`Select` and `Move` are echoes until the engine is connected).